CableTwin

Technical Data Room — Phase 2 · The Build

Automate or Die 2026 · Industry · Production Planning & Supply Chain Optimization

Team SUPCOM · Oussama Akir · Solo participant

The Waze of the cable factory

All companies, data and operational results are fully synthetic.

CableTwin — Technical Data Room

Team: SUPCOM — Oussama Akir (solo) · **Phase 2 — The Build** · All operational results are **synthetic**.

Verify in 3 commands

```
npm run check            # 9/9 automated checks
npm run benchmark:exact  # 17,856 candidates -> 10,440 feasible, optimum confirmed
npm run dev              # launch the offline prototype at http://127.0.0.1:4173/
```

Documents

#	Document	Answers
01	Problem, process and scope	What breaks when a line stops, who decides
02	Product and user journey	Incident -> impact -> 3 strategies -> human approval -> audit
03	AI planning and explainability	Symbolic constraint planning, exact bounded verifier
04	Data model, constraints and assumptions	Data dictionary, synthetic scenario, cost coefficients
05	Validation, results and reproducibility	9/9 tests, exact benchmark, metric formulas
06	Business case, pilot and viability	Buyer, read-only pilot, success gates
07	Risks, security, responsible AI and limits	What can go wrong, human authority
08	Architecture and deployment	Offline-first stack, deployment boundary

Evidence

- `evidence/screenshots/` — frozen prototype screens
- `evidence/test-results/` — test suite output
- `evidence/benchmark-results/` — exact benchmark output

01 — Problem, process and scope

The decision that cannot wait

> *When a production line becomes unavailable, which orders should be moved, delayed or protected — and what will each choice do to service, throughput, overtime and workshop stability?*

CableTwin addresses **incident-driven production rescheduling and capacity reallocation** in a multi-line cable factory. The active production schedule stops being feasible as soon as a required line becomes unavailable. Orders then compete for the remaining compatible capacity while the production manager must protect delivery commitments without creating a second operational problem.

This is a direct implementation of **Industry — Theme 3: Production Planning and Supply Chain Optimization**, specifically AI-assisted production scheduling and capacity optimization. The line stoppage is an input to the workflow; predictive maintenance is not part of the claim.

The incident-driven rescheduling process

The existence of this decision process follows from the operational event: if an order was planned inside a newly unavailable capacity window, the plan must be reassessed. CableTwin does not claim to know how any named manufacturer performs that reassessment today. The local tools, frequency, duration and cost of the process remain pilot questions.

Step	Business event or action	Decision evidence
1	A line becomes unavailable	Line, start time and estimated duration
2	The active schedule becomes infeasible	Orders overlapping or downstream of the stop
3	Orders compete for remaining capacity	Eligible lines, durations and occupied slots
4	Delivery commitments are exposed	Due times, priorities and projected delay
5	Feasible recovery slots are searched	Availability, compatibility, non-overlap and horizon
6	Alternatives are compared	Service, delay, overtime, output, moves and illustrative cost
7	The production manager approves one route	Selected plan and visible trade-off
8	The decision is recorded	Actor, timestamp, strategy and metric snapshot

The process is not “optimize the entire factory.” It is one concrete decision loop triggered by one disruption:

```
Active schedule -> Line stop -> Impact -> Feasible routes -> Human approval -> Audit
```

Who decides: the production manager

The primary user is the **production manager or planner**. This person is responsible for protecting commitments while keeping the revised plan operationally acceptable.

Role	Need in this process
Production manager / planner	Understand impact, compare routes and approve a recovery plan
Workshop supervisor	Receive a clear, feasible revised sequence
Plant or operations director	Review service, capacity and disruption consequences
Customer service	Know which commitments remain threatened

Role	Need in this process
Industrial engineering / IT	Validate constraints, data definitions and integration boundaries

CableTwin supports the manager's expertise; it does not replace responsibility or apply a plan autonomously.

The fixed demonstration incident

The prototype uses one deliberately small, reproducible scenario:

- a **fictional Tunisian cable factory**;
- **3 production lines**, **10 orders** and **241 km** of total ordered output;
- a nominal schedule with **10/10 orders on time**;
- Line 2 stops at **10:00** for **4 hours**, until 14:00;
- 2 orders overlap the stop and 1 downstream order is exposed;
- keeping every order on its original line produces **620 minutes of cumulative delay**, **180 minutes of overtime**, **7/10 orders on time** and **188 km completed by 18:00**.

All companies, customers, products, durations, costs and results are synthetic. The scenario is a controlled test case, not a digital copy of a real plant.

What the working prototype proves

The executable build proves that, for the fixed scenario, CableTwin can:

- identify the orders exposed by the incident;
- generate three complete schedules that respect the encoded hard constraints;
- calculate the same KPI definitions for every alternative;
- preview a revised Gantt before action;
- require explicit human approval;
- record the approved choice; and
- reset deterministically to the nominal state.

These are product and calculation proofs. They are not claims of observed industrial performance.

Scope and boundaries

In scope

- one workshop, one shift and one unplanned line stop;
- complete orders represented as single production slots;
- product-to-line eligibility and line-specific duration;
- due times, priorities, overtime, line moves and illustrative comparison cost;
- three transparent decision policies;
- local, offline, read-only decision support;
- human preview, approval and audit.

Out of scope

- predicting failures or the duration of a failure;
- controlling a machine, PLC or production line;
- writing into an ERP, MES or APS;
- replacing the official production planning system;
- modelling cable physics or a 3D factory;
- multi-stage routing, setup matrices, material stocks, labour, energy or maintenance resources;

- claiming factory-scale global optimality;
- claiming savings, ROI, traction or affiliation with a real manufacturer.

What still requires pilot validation

Status	What is known	What is not yet known
Executable fact	The prototype produces deterministic, feasible routes for the encoded scenario	Whether the encoded constraint set is complete for a target workshop
Synthetic result	Service reduces simulated delay from 620 to 140 minutes	The improvement achievable on real incidents
Product hypothesis	A three-route comparison can shorten and clarify the decision	The current time-to-assess and adoption behaviour at a plant
Data hypothesis	A read-only order/calendar export can supply the minimum model	Actual data availability, quality and refresh process
Commercial hypothesis	A plant or operations director could sponsor a low-risk pilot	Budget, procurement path, pricing and willingness to pay

The next proof is therefore a **4-to-6-week read-only shadow pilot**, not an autonomous deployment.

Integrity disclosure

CableTwin is not affiliated with Chakira Cables, COFICAB, Elloumi Group or any other cable manufacturer. Publicly visible Tunisian cable manufacturing inspired the sector choice only. No operational data, internal process or validation is attributed to a real company.

02 — Product and user journey

Product promise

> *CableTwin lets a production manager see the consequences of a line stoppage, compare three feasible recovery routes and approve one before the real workshop changes.*

CableTwin is a lightweight **incident-time decision twin**. It maintains a calculable representation of the relevant production state, injects a disruption, evaluates recovery schedules and exposes their business trade-offs. The product is designed around one decision, not around a general-purpose dashboard.

Inputs and outputs

Inputs	Outputs
Lines, calendars and availability	Orders directly and indirectly exposed
Orders, quantities, due times and priorities	Three feasible recovery schedules
Product-to-line eligibility and duration by line	Comparable KPI for each route
Current schedule	Revised Gantt preview
Incident line, start and estimated end	Human approval and decision record
Illustrative cost coefficients	Explicit assumptions and comparison values

The complete user journey

Stage	What the user sees or does	What CableTwin produces	Frozen evidence
1. Understand	Opens the nominal workshop: 3 lines, 10 orders, 10/10 on time	A shared view of the initial schedule and assumptions	Nominal factory
2. Trigger	Simulates a 4-hour stop on Line 2 at 10:00	A deterministic incident state	Incident state
3. Measure	Reads 3 exposed orders, 7/10 on time and 620 minutes of projected delay without adaptation	A consistent “no rescheduling” reference	Incident KPI
4. Compare	Reviews Service, Cost and Stability	Three routes built from the same inputs and hard constraints	Strategy cards
5. Preview	Selects a route and inspects the revised schedule	Updated Gantt, affected orders and KPI before commitment	Service preview
6. Approve	Confirms the route as production manager	An approved plan; nothing is sent to a machine	Decision preview
7. Audit	Reviews who chose what and when	Timestamped decision record with strategy and metric snapshot	Approved audit
8. Reset	Restarts the demonstration	Exact return to the 10/10 nominal state	Deterministic reset

The frozen browser capture also records **zero external network requests** for the journey. See `ui-metrics.json`.

The three routes in user language

Route	User-facing question	Fixed-scenario result
Service	How do we best protect priority-weighted delivery commitments?	8/10 on time, 140 min delay, 30 min overtime, 3 line moves
Cost	How do we minimize the illustrative total comparison cost?	8/10 on time, 170 min delay, 60 min overtime, 2 line moves
Stability	What happens if we avoid line reassignments?	7/10 on time, 620 min delay, 180 min overtime, 0 line moves

No route is described as universally best. CableTwin makes the consequence of each business priority visible so the manager can choose the route appropriate to the incident.

Why this is a digital twin

The prototype contains the smallest useful digital representation for this decision:

- an explicit state of lines, orders, calendars and schedule;
- an event that changes resource availability;
- a simulator that recomputes feasible production states and consequences;
- a human decision that selects one future state.

It is intentionally not a high-fidelity physical or 3D twin. The originality is to use the twin where it changes a concrete operational decision at low integration cost.

End-to-end acceptance evidence

The build is considered end to end because the same executable path connects:

```
Scenario data
  -> incident analysis
  -> three constrained plans
  -> KPI calculation
  -> Gantt preview
  -> explicit approval
  -> audit event
  -> deterministic reset
```

The values displayed in the interface are generated from the engine, not typed into presentation cards. The current suite verifies the engine, the bounded exact benchmark verifies the three displayed policy optima, and the Chromium capture verifies the rendered journey.

Product boundaries

CableTwin currently:

- **does** simulate, compare, explain, preview and record;
- **does not** predict the line stop or its duration;
- **does not** command equipment;
- **does not** modify an ERP, MES or APS;
- **does not** learn from industrial history;
- **does not** remove the production manager from the decision;
- **does not** use real factory or customer data.

The first industrial version remains read-only. Export or write-back would be a later, separately governed step after constraint validation and measured pilot success.

Language choice

The jury-facing deck, data room, video narration and captions are in English. The frozen operator interface remains in French because it targets the local operator context and because changing a verified build after the freeze would add avoidable risk. Labels, numbers and interactions are explained in English in every submitted narrative artifact.

03 — AI planning and explainability

The core AI is the planning decision

CableTwin uses **symbolic, constraint-based AI planning** to transform a broken production schedule into feasible recovery alternatives. The intelligence is not a chatbot and it is not the dashboard: it is the structured search over line assignments, order sequences and time slots under industrial rules.

An interface alone can display that Line 2 stopped. The planning engine must answer the harder question:

> *Which complete schedules remain executable, and how do their consequences change when service, cost or workshop stability is the priority?*

Formal decision model

For every order that was not complete when the incident began, the model chooses:

- an eligible production line;
- a position in that line's sequence;
- a start and end time consistent with the line-specific duration.

Hard constraints: a route is rejected if any one fails

- Every order appears exactly once.
- An order is assigned only to an eligible line.
- Its scheduled duration equals its duration on that line.
- A line runs no more than one order at a time.
- No order overlaps the Line 2 stoppage.
- Every order finishes inside the planning horizon.
- Orders already complete at 10:00 stay fixed.

These rules encode feasibility. The prototype never trades a hard constraint for a better score.

Soft objectives: feasible routes can express different priorities

Policy	Lexicographic business objective in the exact verifier	Meaning
Service	Priority-weighted delay -> total delay -> illustrative cost -> overtime -> moves	Protect the most important delivery commitments first
Cost	Illustrative total cost -> total delay -> priority-weighted delay -> moves	Limit the encoded cost of production, delay, overtime and reassignment
Stability	Moves -> total delay -> illustrative cost -> priority-weighted delay	Preserve original line assignments before optimizing consequences

Lexicographic objectives make the priority order explicit and reproducible. They avoid a hidden composite score whose weights a manager cannot interpret.

Two independent layers of evidence

1. Live decision engine

`engine/twin-engine.js` contains the dependency-free planner used by the browser. It is a deterministic constructive search:

- freeze orders complete before the incident;
- order the remaining seven orders according to the selected policy;

- inspect each eligible line;
- find the earliest slot that respects occupied capacity and downtime;
- compare feasible candidates according to the policy;
- build a complete schedule and calculate its KPI.

This implementation is fast, explainable and reliable for the demonstration. It does not claim general factory-scale optimality.

2. Independent exhaustive verifier

`engine/exact-benchmark.js` does not reuse the live planner's construction logic. For the fixed bounded scenario, it enumerates every eligible-line assignment and every within-line sequence for the seven orders to replan, then executes each sequence at the earliest feasible time.

Verification fact	Reproducible result
Assignment-and-sequence candidates evaluated	**17,856**
Feasible complete schedules	**10,440**
Displayed policies checked	**3/3**
Displayed plans matching their policy optimum	**3/3**
Unique policy optima	**3/3**

The exact claim is deliberately bounded to the encoded demonstration: eligible line assignments, within-line sequences, earliest-start execution, the incident window and the planning horizon. It is not a claim that an arbitrary industrial plant can always be solved exhaustively.

Why the search matters

The fixed scenario contains a visible trade-off that a simple “maximize the number of on-time orders” rule would hide:

- the Service policy produces **8/10 on time**, **140 minutes total delay** and **360 priority-weighted delay minutes**;
- a separate 9/10 alternative exists, but produces **230 minutes total delay** and **690 priority-weighted delay minutes**.

The planner therefore protects the severity and priority of lateness, not only a headline count. The independent oracle confirms that this is the unique optimum for the declared Service policy in the bounded scenario.

The other baseline is equally important: avoiding every line reassignment preserves workshop stability but leaves **620 minutes of delay** and only **188 km** completed by 18:00. Constraint-based reallocation is what allows Service to reach **140 minutes** and **216 km**.

Explainability by construction

Every recommendation can be inspected at four levels:

- **Inputs:** orders, priorities, eligible lines, durations and incident window.
- **Rules:** the seven hard constraints above.
- **Policy:** the ordered business objectives for Service, Cost or Stability.
- **Consequences:** orders on time, cumulative delay, overtime, line moves, output and illustrative cost.

The manager sees the revised Gantt before approval. The approved strategy, actor, timestamp and metric snapshot are stored in the audit trail.

Human approval is a design principle

The engine proposes; the production manager decides. Approval is a separate, explicit operation and does not mutate the proposed plan. The current build has no connection capable of commanding a machine or modifying a production system.

This boundary mitigates automation bias and makes an incomplete model safer: the planner can reject, question or override a route before it affects the workshop.

What is AI today, and what could be learned later

Layer	Phase 2 status	Possible pilot evolution
Constraint model	Working and explicit	Add planner-validated setup, material, labour and precedence rules
Decision planning	Working and deterministic	Replace or complement constructive search with CP-SAT at larger scale
Exact evaluation	Working for the bounded demo	Use benchmark suites and optimality gaps for larger cases
Incident-duration prediction	Not claimed	Estimate a duration distribution only if validated history exists
Natural-language explanation	Not required	Optional wording layer; never responsible for schedules or KPI

No machine-learning model is claimed without training data. The current AI value comes from constrained decision search, explicit multi-objective reasoning and verifiable alternatives.

04 — Data model, constraints and assumptions

Data classification

> ***SYNTHETIC DEMONSTRATION DATA.*** *Every company, customer, order, quantity, duration, cost coefficient and operational result in the Phase 2 build is fictional.*

The canonical source is `engine/factory-data.js`. Keeping the dataset in one executable module prevents presentation values from drifting away from the prototype.

Data dictionary

Production line

Field	Type / unit	Purpose
`id`, `name`	text	Stable identity and user-facing label
`normalEndMinute`	minutes from 08:00	End of the normal shift
`productionCostDtPerMinute`	synthetic DT/min	Illustrative production comparison coefficient
`overtimePremiumDtPerMinute`	synthetic DT/min	Illustrative overtime comparison coefficient

Production order

Field	Type / unit	Purpose
`id`, `customer`, `product`	text	Order identity and readable context
`quantityKm`	kilometres	Shift-end throughput calculation
`priority`	integer	Relative service importance
`dueMinute`	minutes from 08:00	Delivery completion target in the scenario
`delayCostDtPerMinute`	synthetic DT/min	Illustrative lateness comparison coefficient
`eligibleLineIds`	line IDs	Product-to-line compatibility
`durationByLine`	minutes by eligible line	Line-dependent production duration

Schedule entry

Field	Type / unit	Purpose
`orderId`, `lineId`	IDs	Assignment decision
`startMinute`, `endMinute`	minutes from 08:00	Planned production interval
`state`	text	Completed or planned after the incident
`movedFromLineId`	line ID or null	Trace of a reassignment

Incident, clock and settings

Object	Fields used
Incident	line, type, start, end, detection time and synthetic reason
Clock	day start, normal shift end, planning horizon and fixed generation time
Settings	synthetic cost of one line reassignment

Fixed scenario

Fact	Canonical value
Factory	Fictional Tunisian cable factory
Day start	08:00
Normal shift end	18:00
Planning horizon	22:00
Production lines	3
Orders	10
Total ordered quantity	241 km
Nominal schedule	10/10 orders on time
Incident	Line 2 unavailable from 10:00 to 14:00
Directly interrupted orders	OF-106 and OF-107
Downstream exposed order	OF-108
Orders replanned by the engine	7; 3 completed orders stay fixed

Time is stored as minutes from the 08:00 day start. This makes every duration, overlap and due-time calculation deterministic while the interface displays human-readable clock times.

Encoded hard constraints

A recovery schedule is valid only if:

- it contains all 10 orders exactly once;
- each order uses an eligible line;
- its duration matches the chosen line;
- start is before end and both remain non-negative;
- no two orders overlap on one line;
- no order overlaps the Line 2 stop;
- no order finishes after the 22:00 planning horizon.

Orders completed by 10:00 remain fixed during recovery construction. The nominal pre-incident schedule is validated without applying the future incident window; all recovery schedules enforce it.

KPI and cost assumptions

The total illustrative comparison cost is:

```
sum (duration x line production coefficient)
+ sum (delay x order delay coefficient)
+ sum (overtime x line overtime coefficient)
+ (number of line moves x 75 DT)
```

The coefficients make alternatives comparable inside one synthetic experiment. They are **not quotations, accounting values, penalties from a customer, factory savings or commercial ROI**.

Two representations appear in the product evidence:

- the benchmark reports absolute synthetic total cost: Service 1,971.15 DT; Cost 1,931.45 DT; Stability 3,862.20 DT;
- the UI cards report rounded incremental cost versus the nominal 1,132.80 DT plan: Service +838 DT; Cost +799 DT; Stability +2,729 DT.

Both are mathematically consistent, but they must never be mixed without the reference point.

Model assumptions

The MVP assumes:

- each order occupies one continuous production slot;
- eligible lines can substitute for each other at the encoded durations;
- order quantity is completed at the end of the slot;
- the incident end is known and deterministic;
- no voluntary idle time improves the declared bounded-demo objectives;
- no new order, material shortage or second failure arrives during planning;
- line and order data are internally consistent.

These assumptions make the demonstration inspectable. They are not assertions about a real cable plant.

Industrial constraints not yet modelled

A real pilot must determine whether to add:

- multi-stage routing and precedence between conductor preparation, insulation, assembly, sheathing, testing and winding;
- sequence-dependent changeovers, cleaning and setup crews;
- tooling, reels, dies, labour qualifications and shared resources;
- material, work-in-progress and packaging availability;
- minimum batch sizes, split lots and campaign constraints;
- quality holds, maintenance windows and energy limits;
- uncertainty ranges for processing and repair duration;
- multiple shifts, days, sites and transport commitments.

No route should be labelled feasible in a pilot until the production planner has validated the relevant rule set.

Proposed read-only pilot data contract

The minimum contract can be supplied as CSV exports, database views or an API, but the first integration remains read-only.

Dataset	Minimum fields	Validation owner
Lines and calendars	ID, availability, shift windows, compatible families	Production planner
Orders	ID, family, quantity, due time, priority/status	Planning / customer service
Line eligibility	Product family, eligible line and effective date	Industrial engineering
Durations	Order or family, line, expected duration and unit	Methods / planner
Active schedule	Order, line, planned start and end	Planning system owner

Dataset	Minimum fields	Validation owner
Incident	Resource, observed start, expected duration or range	Workshop / maintenance
Actual decision	Chosen assignment, decision time and reason	Production manager
Actual outcome	Completion time, overtime, moves and exceptions	Pilot analyst

Validation gates before replay

- Schema and units are agreed.
- Missing, duplicated and impossible records are reported.
- Compatibility and calendar rules are signed off by the planner.
- Historical outcomes are kept separate from recommendations.
- Every replay uses a versioned snapshot and constraint set.
- Data is minimized and customer names can be pseudonymized.

The pilot evidence pack must preserve the lineage from source snapshot to recommendation, approval and KPI result.

05 — Validation, results and reproducibility

Evidence strategy

CableTwin validates the same fixed scenario at three independent levels:

- **engine tests** verify data integrity, constraints, KPI, policy behaviour and approval;
- an **exhaustive bounded oracle** independently certifies the displayed policy optima;
- a **real Chromium replay** verifies that the user can complete the journey and that the rendered values match the engine.

All results below are synthetic demonstration results. They prove calculation and execution, not industrial ROI.

9/9 automated checks

Run:

```
npm run check
```

The frozen build passes **9 tests out of 9** with no syntax error. The suite is defined in `tests/twin-engine.test.mjs`.

#	Automated check
1	Dataset is explicitly synthetic and contains 3 lines / 10 orders
2	Incident identifies OF-106 and OF-107 as directly affected and OF-108 downstream
3	Service, Cost and Stability are deterministic and hard-constraint feasible
4	The three policies produce distinct, understandable business trade-offs
5	The exact oracle certifies all three displayed schedules on the canonical scenario
6	Delay, overtime, cost, line moves, on-time rate and shift-end output are calculated
7	Human approval adds an audit event without mutating the proposed plan
8	Scenario minutes render as readable clock times
9	Nominal metrics calculate before the incident while recovery validation still blocks the stop window

The ninth test is a regression check for the distinction between a valid pre-incident baseline and a valid post-incident recovery schedule.

Independent exact benchmark

Run:

```
npm run benchmark:exact
```

Expected output:

Fact	Result
Orders complete and fixed at incident time	3
Orders to replan	7
Eligible assignment / sequence candidates	**17,856**
Feasible complete schedules	**10,440**
Service result matches its policy optimum	Yes — unique
Cost result matches its policy optimum	Yes — unique

Fact	Result
Stability result matches its policy optimum	Yes — unique

The stored console evidence is `exact-benchmark-output.txt`. The implementation and proof boundary are documented in `engine/exact-benchmark.js`.

The verifier is exact only for the encoded bounded demo: eligible-line assignments, within-line sequences, earliest-start execution, incident window and planning horizon.

Canonical results

Scenario	On-time	Total delay	Overtime	Line moves	Output by 18:00	Synthetic total cost
Nominal, before incident	10/10	0 min	0 min	0	241 km	1,132.80 DT
Stability / no reassignment	7/10	620 min	180 min	0	188 km	3,862.20 DT
Service	8/10	140 min	30 min	3	216 km	1,971.15 DT
Cost	8/10	170 min	60 min	2	216 km	1,931.45 DT

Compared with Stability after the same incident, Service produces:

- **480 fewer delay minutes**, a **77.4%** reduction;
- **150 fewer overtime minutes**, an **83.3%** reduction;
- **28 km more output by 18:00**, a **14.9%** increase;
- one additional on-time order, from 7/10 to 8/10;
- three line reassignments instead of zero.

These are reproducible comparisons inside the fixed synthetic model. They are not forecasts of improvement at a real factory.

UI cost values versus benchmark cost values

The UI rounds and displays the additional synthetic cost versus the nominal plan:

Route	UI incremental value	Benchmark absolute value
Service	+838 DT	1,971.15 DT
Cost	+799 DT	1,931.45 DT
Stability	+2,729 DT	3,862.20 DT

The values share the same nominal reference of 1,132.80 DT. Neither column is real accounting data, savings or ROI.

KPI definitions

For each order `i`, with completion time `C_i` and due time `D_i`:

```
order delay_i       = max(0, C_i - D_i)
total delay         = sum(order delay_i)
on-time orders      = count(C_i <= D_i)
overtime per slot   = max(0, end - max(start, normal shift end))
line moves          = count(recovery line != nominal line)
shift-end output    = sum(quantity for orders completed by 18:00)
```

The illustrative total cost is:

```
production duration cost
+ delay coefficient × delay
```

```
+ overtime premium × overtime
+ 75 DT × line moves
```

Every route is evaluated with the same formulas and data.

A useful counter-example

The oracle finds a separate plan with **9/10 orders on time**, but it creates **230 minutes of total delay** and **690 priority-weighted delay minutes**. The Service result has 8/10 on time, **140 total** and **360 priority-weighted** delay minutes.

This confirms that the declared Service objective protects the severity and priority of lateness rather than optimizing a single headline count.

Browser and end-to-end verification

The full interaction was replayed in a 1920×1080 Chromium session:

```
nominal 10/10
  -> Line 2 incident
  -> 3 exposed orders / 620 min
  -> three strategy cards
  -> Service Gantt preview
  -> human approval at 10:07
  -> audit event
  -> reset to nominal 10/10
```

The capture script recorded **no external network request**. Machine-readable values are stored in `ui-metrics.json`, and the final screenshots are in `evidence/screenshots/`.

Reproduce in under three minutes

Prerequisite: Node.js 22 or a recent compatible version.

```
npm run check
npm run benchmark:exact
npm run dev
```

Open http://127.0.0.1:4173/ and follow:

- verify the nominal 10/10 state;
- click the Line 2 incident trigger;
- compare Service, Cost and Stability;
- preview Service;
- approve the plan;
- verify the audit entry;
- reset and confirm 10/10.

To reproduce the evidence capture while the server is running:

```
node scripts/capture-screens.mjs
```

What has and has not been validated

Validated in Phase 2	Not yet validated
Executable end-to-end workflow	Real plant workflow and user adoption
Encoded hard-constraint feasibility	Completeness of constraints for a target workshop
Deterministic KPI calculations	Accuracy of real durations and incident estimates
Unique policy optima in the bounded demo	Optimality or runtime at factory scale

Validated in Phase 2	Not yet validated
Offline browser operation	ERP/MES data integration
Human approval and audit event	Production governance, roles and access control
Synthetic before/after comparisons	Industrial savings, ROI or commercial demand

Known limitations

- One fixed incident is evidence of mechanism, not statistical performance.
- All input values and costs are synthetic.
- Each order is modelled as one continuous operation.
- Incident duration is known rather than probabilistic.
- The live planner is constructive; the exact certificate is bounded to this small scenario.
- There is no persistent database, authentication, ERP write-back or machine connection in the prototype.

06 — Business case, pilot and viability

Business proposition

CableTwin is not sold as “AI for the factory.” It addresses one event-driven decision:

> *After an unplanned line stoppage, compare feasible recovery schedules before changing production.*

The product can create value by shortening the time needed to assess an incident, protecting delivery and shift-end output, reducing avoidable overtime and movements, and making the chosen response traceable. Which value dominates is plant-specific and must be measured.

User, buyer and beneficiaries

Role	Relationship to CableTwin	Value sought
Production manager / planner	Primary user and final operational decision-maker	Faster, feasible and explainable recovery options
Plant or operations director	Pilot sponsor and likely economic buyer	Service, throughput and disruption evidence
Workshop supervisor	Execution stakeholder	A clear revised sequence that respects workshop rules
Customer service	Downstream beneficiary	Earlier visibility of threatened commitments
Industrial engineering / IT	Technical prescriber and integrator	Explicit constraints, low-risk data access and auditability
Finance / management	Evidence consumer	Transparent assumptions rather than invented savings

The initial beachhead is a multi-line cable or electrical-equipment workshop where some product families can move between lines and delivery commitments matter.

Measurable value in the demonstration

The fixed synthetic incident shows the mechanism:

Outcome	Stability after incident	Service route	Difference
Cumulative delay	620 min	140 min	-480 min / -77.4%
Overtime	180 min	30 min	-150 min / -83.3%
Output completed by 18:00	188 km	216 km	+28 km / +14.9%
On-time orders	7/10	8/10	+1 order
Line reassignments	0	3	+3 moves

This table demonstrates that the system can calculate and expose a trade-off. It is not a claimed factory result.

A low-risk 4-to-6-week shadow pilot

The first commercial step is one workshop, one decision process and no write access.

Stage	Activity	Evidence produced
1. Discover	Confirm the decision owner, current workflow, unacceptable errors and KPI definitions	Signed process and KPI sheet
2. Contract data	Map read-only orders, schedules, calendars, compatibilities and incidents	Data dictionary and quality report
3. Configure	Encode planner-validated constraints and reproduce one known schedule	Constraint acceptance checklist
4. Replay	Re-run 5–10 historical stoppages without changing their historical outcome	Baseline-versus-CableTwin replay pack
5. Shadow	Run beside the current process on qualifying incidents; manager remains in control	Decision-time, feasibility and usability log
6. Decide	Review pre-agreed gates with the sponsor	Go, revise or stop decision

The historical decision remains the baseline. CableTwin receives the same available information cutoff and uses identical KPI definitions; later outcomes must not leak into the replay.

Proposed pilot success gates

These are proposed thresholds to agree with the pilot sponsor, not achieved claims:

Gate	Definition
30% faster assessment	Median incident-to-comparable-options time versus the documented current process
20% lower projected or replayed delay	Cumulative delay versus the actual historical decision, using the same due-time definition
Zero validated-constraint violations	No recommended plan breaks a hard rule signed off by the planner
At least 70% planner usability	Share of evaluated incidents where the planner rates at least one route usable or useful, with reasons recorded
100% decision traceability	Every recommendation stores input version, constraint version, route, KPI, actor and timestamp

Plant-specific service, output, overtime and movement KPI can replace or refine these gates before the pilot begins.

Pilot inputs and resources

Minimum resources:

- one production manager or planner as business owner;
- one industrial-engineering or IT contact;
- read-only historical exports for 5–10 stoppages;
- line calendars, product-line eligibility, line-specific durations, due times and priorities;
- the actual decision and outcome for each replay;
- one local workstation or controlled internal server;
- a weekly 30-minute review during the pilot.

No sensor project, GPU, paid API, cloud migration, machine connection or ERP write permission is needed for the first proof.

Business viability path

```
Hackathon proof
  -> fixed-scope shadow pilot
```

```
-> measured go/no-go
-> paid configuration and integration
-> per-site software subscription hypothesis
-> optional multi-workshop expansion
```

No price, signed customer, revenue, annual savings or market share is claimed. Commercial terms are discussed only after the pilot defines integration effort, support needs and measurable value.

Competitive differentiation

Alternative	Strength	CableTwin's complementary difference
Spreadsheet, calls and planner experience	Familiar, flexible and already available	Adds repeatable constraint checks, comparable routes and an audit trail while preserving human judgement
ERP / MES	Trusted system of record and execution context	Sits beside it as a focused read-only incident-decision layer
APS / optimization suite	Broad planning depth and mature integrations	Starts with one explainable disruption workflow and a lower-risk shadow pilot
High-fidelity digital twin	Rich physical or process representation	Uses the smallest useful decision twin; no 3D model or full plant digitization required

CableTwin should not replace a capable APS that already performs this workflow well. Its entry point is a plant where incident response remains manual, difficult to compare or weakly traced.

Defensible product asset

The defensible asset is not the synthetic demonstration dataset. It would be the combination of:

- a planner-validated constraint model for the target workshop;
- versioned incident and decision history;
- consistent KPI definitions;
- feedback on which routes were feasible, chosen and successful;
- integration into the actual approval workflow.

That asset improves with each validated incident while remaining explainable.

Commercial and operational stop conditions

The pilot stops or is redesigned if:

- the target process does not require alternative schedules;
- the necessary data cannot be obtained reliably in read-only form;
- real constraints cannot be represented or validated;
- no plan can move between compatible capacity;
- recommendations repeatedly fail planner review;
- the product does not improve a pre-agreed KPI;
- security or governance requirements cannot be met.

This go/no-go discipline protects the customer from an attractive demonstration that does not translate into operational value.

07 — Risks, security, responsible AI and limits

Responsible deployment principle

A schedule is useful only if its data, constraints and authority are trusted.

CableTwin is designed as decision support. The current build works offline, uses synthetic data, has no production-system write access and requires explicit human approval. A pilot would preserve these boundaries until its model and value are validated.

Risk register

Risk	Impact	Current control	Required pilot control
Missing or wrong hard constraint	A route appears feasible but cannot be executed	Explicit encoded rules and automatic validation	Planner sign-off, versioned rule set, constraint acceptance tests
Incorrect incident duration	Recovery plan becomes stale or misleading	Duration is visible and fixed in the scenario	Duration range, update trigger, replan when estimate changes
Poor or stale source data	Wrong assignments, due times or KPI	Single controlled synthetic source	Read-only snapshots, freshness marker, schema and quality checks
Cost model mistaken for ROI	Misleading business claim	All coefficients labelled synthetic	Customer-approved definitions; keep operational and accounting evidence separate
Planner over-trusts the recommendation	Automation bias or unreviewed execution	Three routes, visible trade-offs, explicit approval	Training, reason codes, override, no automatic write-back
Incomplete explainability	Manager cannot defend the choice	Inputs, policy, Gantt and KPI are visible	Per-order reason and constraint trace where required
Production data disclosure	Commercial or customer confidentiality breach	Local app and synthetic dataset	Least-privilege access, pseudonymization, retention policy and internal hosting
ERP/MES integration error	Planning disruption or data corruption	No integration in Phase 2	Read-only adapter first, reconciliation, staged export and rollback
Search does not scale	Slow response on industrial volumes	Small deterministic scenario; bounded exact proof	Performance benchmark, timeout, solver evaluation and safe fallback
Model drift after plant changes	Recommendations use obsolete capability	Static versioned demo	Owner, effective dates, change approval and periodic revalidation
Low workshop adoption	Useful routes are ignored or misunderstood	Plain-language policies and French operator UI	Co-design, usability log, champion and feedback loop

Risk	Impact	Current control	Required pilot control
Prototype availability failure	Decision support unavailable during incident	Offline build and standalone archive	Supported internal deployment, monitoring and documented manual fallback

Human authority and safe failure

CableTwin separates four states:

- **calculated** — a route exists in the model;
- **validated by rules** — it satisfies the encoded hard constraints;
- **reviewed by a manager** — a human checks operational context;
- **approved** — the manager accepts responsibility for the proposed route.

The prototype implements the first, second and explicit simulated approval states. It does not send the approved route to a machine or production system.

If data is missing, constraints are unresolved or no feasible route exists, the safe behaviour is to stop and escalate to the planner — never to relax a hard constraint silently.

Responsible AI controls

Principle	CableTwin implementation
Transparency	Synthetic data notice, explicit policies, formulas and limitations
Validity	Constraint validation, 9/9 automated checks and independent bounded oracle
Human oversight	Preview and named approval before any decision is recorded
Contestability	Three routes and visible trade-offs; the manager may reject all
Traceability	Scenario, strategy, metrics, actor and timestamp in the audit event
Proportionality	One bounded operational decision; no unnecessary LLM or autonomous control
Data minimization	Only scheduling fields required for the decision; customer identity can be pseudonymized
Honest claims	Simulation separated from pilot hypotheses and real-world evidence

Security posture

Phase 2 prototype

- bound to `127.0.0.1`, not exposed as a public server by default;
- works without an external API, cloud call or runtime dependency;
- contains no production credentials or real company data;
- browser evidence recorded zero external network requests;
- no authentication because the prototype is local and synthetic;
- no database, machine connection or ERP/MES write path.

Minimum pilot controls

- deploy inside the customer's approved environment;
- use read-only service credentials and least-privilege database views or file drops;
- encrypt data at rest and in transit where applicable;
- pseudonymize customer or product identifiers where names are not required;

- define retention, deletion, backup and incident-response rules;
- record input and constraint versions for every recommendation;
- add role-based access for configuration, review and approval;
- perform a security review before any export or write-back.

Offline-first reduces exposure but does not remove the need for access control once real data is used.

Uncertainty management

The demonstration knows that Line 2 will be unavailable for four hours. A real repair estimate is uncertain. A pilot should:

- show the estimate, source and last update;
- allow a range or multiple duration scenarios;
- calculate sensitivity of the route to the duration;
- replan when the estimate crosses a defined threshold;
- warn when an approved plan is based on stale incident information.

CableTwin must not claim certainty that does not exist in the source data.

Explicit limitations

- The dataset is fully synthetic and contains one incident.
- No industrial user, customer or signed pilot is claimed.
- The model represents one order as one continuous production slot.
- Setup, labour, tooling, material, quality and energy constraints are absent.
- Processing and incident durations are deterministic.
- The live planner is constructive; exhaustive optimality is certified only for the encoded small scenario.
- The cost model is illustrative and not an accounting or ROI model.
- The prototype is not persistent and has no production identity management.
- No machine, PLC, ERP, MES or APS is controlled or modified.
- Predictive maintenance, failure diagnosis and autonomous execution are outside scope.

Deployment gate

CableTwin should progress beyond read-only shadow mode only if:

- the production planner has accepted the constraint model;
- replay evidence shows zero hard-constraint violations;
- a pre-agreed KPI improves;
- managers understand and can contest recommendations;
- security and data-governance owners approve the architecture;
- an operational fallback and rollback process exists.

Until then, the product remains an advisory layer and the current planning process remains authoritative.

08 — Architecture and deployment

Architecture objective

The Phase 2 build prioritizes a reliable jury demonstration and a low-risk industrial starting point:

- local and offline;
- zero external runtime dependency;
- deterministic calculations;
- explicit separation between data, planning and interface;
- no machine or production-system control.

Current end-to-end architecture

```
engine/factory-data.js
Synthetic lines, orders, schedule, incident and coefficients
                          |
                          v
engine/twin-engine.js
Validation -> incident analysis -> 3 constrained plans -> KPI -> approval event
                          |
                          v
app/app.js + app/index.html + app/styles.css
Nominal state -> incident -> comparison -> Gantt preview -> approval -> audit -> reset
                          |
                          v
Local Chromium browser at http://127.0.0.1:4173/

Independent verification path:
engine/exact-benchmark.js -> exhaustive bounded oracle -> policy-optimum evidence
```

The exact verifier is kept separate from the live constructive planner so it can detect, rather than reproduce, a planning error.

Module responsibilities

Module	Responsibility	Key output
`engine/factory-data.js`	Canonical synthetic dataset and disclosure	Fresh scenario object
`engine/twin-engine.js`	Scenario/schedule validation, incident analysis, route construction, KPI and approval	Three proposed plans and one approved plan
`engine/exact-benchmark.js`	Independent exhaustive evaluation of the bounded demo	Unique policy optima and best on-time alternative
`app/app.js`	UI state and rendering	Interactive decision journey
`app/index.html`	Semantic structure	Browser entry point
`app/styles.css`	Responsive visual system	Projector and mobile layouts
`scripts/serve.mjs`	Dependency-free local HTTP server	`127.0.0.1:4173`
`tests/twin-engine.test.mjs`	Automated regression and acceptance checks	9/9 passing checks
`scripts/capture-screens.mjs`	Real-browser replay through Chromium DevTools Protocol	Screenshots, UI metrics and network evidence

Runtime and dependency profile

Item	Phase 2 choice
Runtime	Node.js 22 or recent compatible version
Front end	Native HTML, CSS and JavaScript modules
Server	Node standard-library HTTP server
Application dependencies	None
External API / model	None
Cloud requirement	None
GPU requirement	None
Network during main journey	None; browser capture recorded zero external requests
Persistent storage	None in the demonstration

This profile makes the prototype easy to launch from a standalone archive and reduces hackathon infrastructure risk.

Data and decision flow

- `createFactoryScenario()` returns a new controlled scenario.
- `validateScenario()` checks IDs, data ranges, compatibility, durations, initial coverage and incident consistency.
- `analyzeIncident()` identifies orders exposed on the stopped line.
- `generateRecoveryPlans()` runs the Service, Cost and Stability policies.
- `validateSchedule()` rejects any hard-constraint violation.
- `calculateMetrics()` derives comparable operational and illustrative cost KPI from each complete schedule.
- The app renders strategy cards and a revised Gantt.
- `approveRecoveryPlan()` returns a new approved object and audit event only after explicit user input.
- Reset creates the original scenario again.

All engine operations are deterministic; decision time and approving identity are explicit inputs rather than hidden system state.

Deployment boundary

What the current build can access

```
Bundled synthetic JavaScript data
              v
Local planning engine
              v
Local browser
```

What it cannot access

- PLCs, sensors, drives or machines;
- production networks;
- ERP, MES or APS databases;
- customer systems;
- external AI or cloud services.

The “Approve” action records a simulated decision in memory. It does not create a production order or transmit a command.

Proposed read-only pilot architecture

```
ERP/MES/CSV read-only export
            |
            v
Schema + quality validation ------> rejected-record report
            |
            v
Versioned workshop snapshot + versioned constraint model
            |
            v
CableTwin planner + KPI + explanation
            |
            v
Production-manager review and approval
            |
            +--------> immutable pilot audit record
            +--------> comparison only; no automatic write-back
```

The historical decision and actual outcome are stored separately for replay evaluation. A later plan export is a new governed capability, not an implicit part of the pilot.

What a real deployment would add

Capability	Reason
Import adapters and schema mapping	Connect approved read-only sources
Persistent versioned store	Preserve inputs, constraints, routes and decisions
Authentication and role-based access	Separate configuration, review and approval
Constraint administration	Manage effective dates and planner sign-off
Solver and performance layer	Support larger workshops and time limits
Uncertainty / sensitivity analysis	Replan safely when incident duration changes
Monitoring and observability	Detect failure, latency, stale data and unusual outputs
Security controls	Encryption, retention, access audit and incident response
Controlled export	Share an approved plan without direct machine control

Non-functional targets for a pilot

- **Feasibility:** zero violations of planner-validated hard constraints.
- **Reproducibility:** same input and rule versions produce the same result.
- **Traceability:** every route retains data, constraint, objective and decision lineage.
- **Availability:** a documented manual process remains available if CableTwin is unavailable.
- **Performance:** a response-time target is agreed after measuring realistic problem size.
- **Security:** least-privilege, read-only access and minimized data.
- **Usability:** a manager can understand the main trade-off without algorithmic vocabulary.

Launch and verify

```
npm run check
npm run benchmark:exact
npm run dev
```

Then open http://127.0.0.1:4173/.

The current evidence set contains the post-fix browser journey at 1920×1080, machine-readable UI values and an offline-network trace under `evidence/screenshots/`.